

# A deep reinforcement learning hyperheuristic for the covering tour problem with varying coverage

Parisa Torabi <sup>a,\*</sup>, Ahmad Hemmati <sup>b</sup>, Anna Oleynik <sup>a</sup>, Guttorm Alendal <sup>a</sup>

<sup>a</sup> Department of Mathematics, University of Bergen, Allégaten 41, Bergen, Norway

<sup>b</sup> Department of Informatics, University of Bergen, Thormøhlens Gate 55, Bergen, Norway

## ARTICLE INFO

### Keywords:

Hyperheuristic  
Adaptive metaheuristic  
Combinatorial optimization  
Deep reinforcement learning

## ABSTRACT

Covering Tour Problem (CTP) is a combinatorial optimization problem in which the objective is to identify a minimum-cost tour that satisfies the coverage of a certain subset of nodes in a graph. The Covering Tour Problem with Varying Coverage (CTP-VC) is an extension of this problem in which the coverage radius is dependent on the amount of time spent at each node. In this paper, we propose a novel approach to address the CTP-VC using a Deep Reinforcement Learning Hyperheuristic (DRLH). This study includes experiments on the existing Adaptive Metaheuristic to solve CTP-VC, to enhance its solution quality. Further, new heuristics and three selection methods, namely Uniform Random Selection (URS), adaptive Metaheuristic (AMH), and the proposed DRLH are introduced. We detail the computational setup, including the instance sets utilized, the training process for the DRLH agent, and the validation procedures for model selection. Through extensive experimentation and analysis, we evaluate the performance of different selection methods, assess the solution quality of the DRLH approach, investigate the robustness of selection methods, examine heuristic selection frequency, and analyze solution convergence. Our results demonstrate the efficacy of the DRLH approach in tackling the CTP-VC, offering promising insights for future research in the interface of combinatorial optimization and reinforcement learning methodologies.

## 1. Introduction

Routing problems are an important class of combinatorial optimization problems that undergo extensive studies across different variants. In particular, the Covering Tour Problem (CTP) (Gendreau et al., 1997) attempts to find a minimum-length tour of a subset of nodes in a graph, such that certain nodes are guaranteed to be visited, and certain others are located within a maximum distance of the visiting tour. This problem has potential applications in bi-level transportation networks in which services can be delivered indirectly to locations, via making stops in “nearby” locations. An example is the optimal design of post-box locations, minimizing a linear combination of the distances traveled by the users and postal service operators (Labbe and Laporte, 1986). Variants of CTP also appear in applications such as surveillance (Oliveira et al., 2015), environmental monitoring (Ouelmokhtar et al., 2022), locating recycling drop-off stations (Cubillos and Wöhlk, 2021), mobile healthcare facilities (Hodgson et al., 1998; Oppong and Hodgson, 1994; Doerner et al., 2007), healthcare prevention teams (Brown and Fintor, 1995), delivery in the dairy industry (Simms, 1989) or disaster relief teams (Kovács and Spens, 2007).

In the classical CTP, the covering distance or covering set of nodes is considered a known parameter in the problem. However, some factors can cause variations in this parameter. For instance, in offshore monitoring, the amount of time spent at each point can vary covering distance. To deal with this variation, the Covering Tour Problem with Varying Coverage (CTP-VC) (Torabi et al., 2023) was introduced. This problem was mathematically modeled and solved using Gurobi, an exact solver, for different instance sets, and it proved ineffective when the size of instances increased. Consequently, an Adaptive Metaheuristic was also designed and tested on the instances, which has shown good performance in a reasonable time. Many aspects of this Adaptive Metaheuristic are inspired by Adaptive Large Neighborhood Search (ALNS) (Stefan Ropke, 2006), in which the objective value is iteratively improved by applying a set of destruction (removal) and reconstruction (insertion) operators on an incumbent solution, and the selection probability of different sub-heuristics is adjusted after a so-called segment has passed. A segment consists of a number of iterations of the algorithm, and the adjustment of probabilities is based on the performance of the sub-heuristics in the preceding segment.

\* Corresponding author.

E-mail addresses: [parisa.torabi@uib.no](mailto:parisa.torabi@uib.no) (P. Torabi), [ahmad.hemmati@uib.no](mailto:ahmad.hemmati@uib.no) (A. Hemmati), [anna.oleynik@uib.no](mailto:anna.oleynik@uib.no) (A. Oleynik), [guttorm.alendal@uib.no](mailto:guttorm.alendal@uib.no) (G. Alendal).

<https://doi.org/10.1016/j.cor.2024.106881>

Received 23 February 2024; Received in revised form 19 June 2024; Accepted 21 October 2024

Available online 28 October 2024

0305-0548/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Since its introduction in 2006, ALNS has been applied to several problems, mostly in various routing problems, see the review paper (Windras Mara et al., 2022). Contributing to this prevalence is the availability of well-performing heuristics for these problems, which can act as operators or sub-heuristics in ALNS. Moreover, ALNS possesses the ability to explore the solution space in a structured manner and is fairly robust.

However, there are shortcomings associated with ALNS. Among others, the adaptive layer of ALNS has a minor effect on the quality of the solutions studied in Turkeš et al. (2021). Moreover, the selection probability of sub-heuristics is only affected by its past performance and does not consider different phases of the search process. The other limitation is what (Kallestad et al., 2023) refer to as “macro-level” decision-making capability, i.e., the probability of each sub-heuristic being chosen at each step is fixed during the iterations in each segment and is updated only at the end of the segment. This makes it impossible for ALNS to take into account the short-term dependencies within a segment and leverage them to guide the search process. When faced with a large number of sub-heuristics, the performance of ALNS declines, and therefore, an initial stage of experiments is required to reduce this number and identify a best-performing set of sub-heuristics. This process might also omit sub-heuristics that can be effective in specific circumstances but perform poorly on average.

Kallestad et al. (2023) strive to address these issues by introducing a general Deep Reinforcement Learning Hyperheuristic (DRLH) framework for solving Combinatorial Optimization (CO) problems. In this framework, the adaptive layer of ALNS is replaced by a Deep Reinforcement Learning (RL) agent. RL is concerned with training an agent to learn a policy through interaction with an environment and observing the reward given to certain actions taken by the agent. This policy is aimed at maximizing the total expected reward. The term Hyperheuristic (Cowling et al., 2001) is defined as “a search method or learning mechanism for selecting or generating heuristics to solve computational search problems”. According to the classification in Burke et al. (2010), DRLH can be classified as a selection hyperheuristic.

Contrary to ALNS, DRLH can make decisions on a “micro level”, i.e., in each iteration of the search, the probabilities are dependent only on the observations of the agent from the environment. This allows for abrupt and significant changes in the probabilities from one iteration to the next if and when new information comes to light. DRLH has shown promising results for some of the most well-known problems in the field of CO, namely the Capacitated Vehicle Routing Problem (CVRP), Parallel Job Scheduling Problem (PJSP), Pickup and Delivery Problem (PDP), and Pickup and Delivery Problem with Time Windows (PDPTW) (Kallestad et al., 2023).

In this study, we leverage the decision-making capability of DRLH to improve the heuristic selection for CTP-VC. We first improved the Adaptive Metaheuristic developed in Torabi et al. (2023), including introducing a new set of heuristics. Moreover, we compared its performance with that of the new DRLH-based method. We include a detailed analysis of the different selection methods and give insights into the differences in performance.

This paper is organized into the following sections: Section 2 gives an overview of related literature, divided into Section 2.1, where the work on CTP and its variants is reviewed, and Section 2.2 in which the focus is on RL-based methods for solving combinatorial optimization problems. Section 3 gives a description and mathematical formulation of CTP-VC. Section 4 gives an overview of the heuristics and different selection methods that are used in this study. In Section 5, a detailed description of computational experiments and their results are given. We conclude the study in Section 6 where we also discuss limitations and future research avenues.

## 2. Literature review

### 2.1. The Covering Tour Problem, its variants, and solve methods

Throughout this section, we will consider the graph  $G = (V \cup W, E)$ . The Covering Tour Problem (CTP) was introduced and formulated (Gendreau et al., 1997) as a generalization of the Covering Salesman Problem (CSP) (Current and Schilling, 1989). CTP can be stated as follows: given  $G$ , find a minimum-length tour of a subset of  $V$  such that every node in  $W$  is within a pre-specified distance of a node on the visiting path. A mathematical model of the problem is given in Gendreau et al. (1997) and a heuristic and a branch-and-cut algorithm are proposed to solve it. The CTP is formulated as a two-commodity network flow problem in Baldacci et al. (2005). The authors developed two scatter-search metaheuristics and compared their algorithms with a classic scatter-search method introduced in Glover (1977).

A bi-objective extension of CTP was introduced in Jozefowicz et al. (2007) as the bi-objective Covering Tour Problem (BOCTP). The authors considered CTP with a different objective, namely the simultaneous minimization of two objective functions: the tour length, and the greatest distance between a covered node  $w \in W$  and its nearest visited node, presumably covering  $w$ . They used a cooperative strategy to solve the problem. It combines a multi-objective evolutionary algorithm that they have tailored to BOCTP and the branch-and-cut algorithm proposed by Gendreau et al. (1997). The Median Tour Problem (MTP) and the Maximal Covering Tour Problem (MCTP) are two similar problems that were put forward by Current and Schilling (1994), prior to CTP. Both problems attempt to find a tour of  $p$  nodes out of  $n$  that simultaneously minimizes total tour length and maximizes access to the tour for the nodes not directly on it. MTP defines the access objective as the minimization of the total demand-weighted travel distance of the non-visited nodes to reach their nearest stop on the tour. In the MCTP, the access objective is to maximize the total demand within some pre-specified maximal travel distance from a tour stop.

A multi-vehicle CTP ( $m$ -CTP) was defined in Hachicha et al. (2000) as a generalization of CTP that seeks a set of total minimum-length vehicle routes on a subset of  $V$ , such that every node in  $W$  is covered by at least one node on a vehicle route and the constraints in Gendreau et al. (1997) are met. Moreover, there is a limit on the number of vertices visited by a vehicle, and the length of each tour. An Integer Linear Program of this problem was given, and three heuristics were developed for this problem on randomly generated instances and a real-life problem associated with the planning of mobile healthcare facilities in Ghana. Later, a branch-and-price algorithm for  $m$ -CTP was developed in Jozefowicz (2014). Authors in Glize et al. (2020) propose an exact method for the  $m$ -CTP based on column generation techniques. The method for the  $m$ -CTP is also embedded in an  $\epsilon$ -constraint exact method to solve the bi-objective version of  $m$ -CTP.

A stochastic BOCTP was studied in Tricoire et al. (2012), where the authors extend the MCTP by changing the binary nature of being covered, to a more general setting in which a percentage of clients travel to a tour stop; this percentage is a decreasing function of the distance to the tour stop. Moreover, the tour stops are taken as distribution centers and have a certain capacity and opening cost. The demand in the population centers is stochastic, and multiple vehicles can be involved in distributing the goods to the distribution centers. In order to solve this problem, authors combine a scenario-based approach with an epsilon-constraint method based on branch-and-cut and demonstrate their algorithm on real-world test instances from rural communities in Senegal.

## 2.2. Reinforcement learning-based solve methods for combinatorial optimization problems

Machine Learning (ML) is one of the fastest-growing fields in recent years, and it has found its way into several areas of research. CO problems are among the ones that have been attempted to be solved using ML approaches. Bengio et al. (2021) have surveyed the attempts to solve CO problems using ML methods. They suggest further integration of ML and CO, by considering generic optimization problems as data points and investigating the relevant distribution of problems to be used for learning on a given task. Learning CO on graphs is studied in Vesselinova et al. (2020), where the authors organize and compare the structures involved in learning to solve CO problems, with a focus on applications in networks. A comprehensive review of methods and applications of Graph Neural Networks (GNNs) is given in Zhou et al. (2021), where the authors propose a general pipeline for designing GNN models and discuss the components and the variants of each component, and categorize the applications.

RL is one of the three basic ML paradigms, together with Supervised and Unsupervised Learning. In short, an agent interacts with an environment, the dynamics of which are not entirely known. The agent attempts to learn these dynamics by taking action in each step and observing the positive or negative reward given by the environment. As a method to guide sequential decision-making, using RL to solve CO problems has attracted a significant amount of attention. Problem-specific algorithms have been developed to solve some of the well-known CO problems. Among others, Bello et al. (2016) present a framework to tackle CO problems using neural networks and RL. They specifically focus on the Traveling Salesman Problem (TSP) and optimize the parameters of a recurrent neural network (RNN) using a policy gradient method with the negative tour length as the reward signal. This RNN is expected to predict the shortest Hamiltonian tour of a weighted graph.

A Deep Reinforcement Learning-based Neural CO strategy is used to solve the online Vehicle Routing Problem (VRP) in Yu et al. (2019) and Li et al. (2021) propose a deep RL approach to solve the CSP. Deep RL is used by Kool et al. (2019) as a constructive heuristic for solving a range of routing problems, where the agent chooses the next node to visit at each step. Nazari et al. (2018) train a policy model that finds near-optimal solutions for VRP instances of similar size by observing the reward signals and following feasibility rules. A learning-based approach for CVRP, “Learn to Improve” (L2I), is proposed in Lu et al. (2019), where L2I learns to iteratively refine a solution with an improvement operator, selected by a RL-based controller. For surveys on the use of RL in solving CO problems, we refer the reader to Yang and Whinston (2020), Wang and Tang (2021) and Mazyavkina et al. (2021).

Using ML in hyperheuristics has gained attention in recent years. Among others, Özcan et al. (2012) investigate the effect of using RL in heuristic selection on the performance of a great-deluge-based hyper-heuristic and show their results on an examination timetabling problem. In Asta and Ozcan (2014) the authors propose Apprenticeship Learning Hyper-Heuristic (ALHH) for VRP, in which an agent is trained to imitate an expert agent through supervised learning. Tyasnurita et al. (2015) have used a multilayer perceptron (MLP) as the apprenticeship learning algorithm to improve upon the performance of ALHH. Although both methods have shown promising results on VRP, they both suffer from the requirement of having an expert behavior (labeled data), and being bounded by the expert’s performance. DRLH improves upon these methods by using RL and eliminating the need for labeled data.

## 3. Problem description

We consider an Autonomous Underwater Vehicle (AUV) that can move and stay in specific points in the ocean environment and take repeated measurements of chemicals, e.g., pH, salinity, and dissolved inorganic carbon (DIC). Our ultimate goal is to identify signals that can indicate a seep of pollutants, such as CO<sub>2</sub> from a storage site. As the pollutant disperses due to the ocean currents, repeated measurements at one location can give indication of a leak at another location. In this er say that this location is *covered*. We also consider the visited locations to be covered. The aim is to cover as much of a specified area as possible while respecting the limited battery capacity of the vehicle. We call this problem the Covering Tour Problem with Varying Coverage (CTP-VC), which is an extension of CTP with profits where the covering distance of nodes depends on the amount of time spent in each node. Here we assume that this distance,  $r$ , increases linearly with the time,  $\tau$ , spent at the node, i.e.,

$$r(\tau) = \begin{cases} \alpha\tau + \beta, & 0 \leq \tau < t_{\max} \\ R, & \tau \geq t_{\max}, \end{cases} \quad R = \alpha t_{\max} + \beta, \quad \alpha, \beta \geq 0, \quad (1)$$

where  $t_{\max}$  is a priori defined as the maximum time that can be spent at a node.

Mathematically put, we consider a simple weighted graph  $G = (V, E)$  where  $V = \{v_i, i = 0, 1, \dots, n, n+1\}$ ,  $n \in \mathbb{N}$ ,  $E \subseteq V \times V$ , and assign non-negative importance  $W_i$  to each node, and a weight  $D_{ij} \geq 0$  to each edge  $(v_i, v_j) \in E$ . The nodes  $v_0$  and  $v_{n+1}$  are the origin and destination of the movement of a vehicle, attempting to maximize a scalarization of the total importance of the covered and visited nodes and the negation of the costs of the operation.

A list of sets, parameters, and variables used in the mathematical model is given in Table 1, and the mathematical model of this problem, developed in Torabi et al. (2023) is given as

$$\text{maximize } \omega \sum_{j \in V \setminus \{v_0, v_{n+1}\}} W_j(y_j + z_j) - \left( \sum_{(i,j) \in E} C_{ij} \sum_{\tau \in T} x_{ij\tau} + \sum_{j \in V \setminus \{v_0, v_{n+1}\}} \hat{C}_j t_j \right) \quad (2)$$

subject to

$$t_j = \sum_{k \in N_j} \sum_{\tau \in T} \tau x_{jk\tau} - \sum_{i \in N_j} \sum_{\tau \in T} (\tau + T_{ij}) x_{ij\tau}, \quad j \in V \setminus \{v_0, v_{n+1}\}, \quad (3)$$

$$\sum_{j \in N_0} x_{0jT_0} = 1, \quad (4)$$

$$\sum_{i \in N_{n+1}} \sum_{\tau \in T} x_{i,n+1,\tau} = 1, \quad (5)$$

$$t_j \geq 0, \quad j \in V \setminus \{v_0, v_{n+1}\}, \quad (6)$$

$$\sum_{(i,j) \in E} x_{ij\tau} \leq 1, \quad \tau \in T, \quad (7)$$

$$\sum_{i \in N_j} \sum_{\tau \in T} x_{ij\tau} - \sum_{k \in N_j} \sum_{\tau \in T} x_{jk\tau} = 0, \quad j \in V \setminus \{v_0, v_{n+1}\}, \quad (8)$$

$$U \sum_{(i,j) \in E} D_{ij} \sum_{\tau \in T} x_{ij\tau} + \hat{U} \sum_{j \in V \setminus \{v_0, v_{n+1}\}} t_j \leq B, \quad (9)$$

$$y_j = \sum_{i \in N_j} \sum_{\tau \in T} x_{ij\tau}, \quad j \in V \setminus \{v_0, v_{n+1}\}, \quad (10)$$

$$r_j - D_{ij} + M_1(1 - s_{ij}) \geq 0, \quad (i, j) \in E, \quad (11)$$

$$\sum_{(i,j) \in E} s_{ij} + M_2(1 - z_i) - 1 \geq 0, \quad i \in V \setminus \{v_0, v_{n+1}\}, \quad (12)$$

$$\sum_{(i,j) \in E} s_{ij} - M_2 z_i \leq 0, \quad i \in V \setminus \{v_0, v_{n+1}\}, \quad (13)$$

$$y_i + z_i \leq 1, \quad i \in V \setminus \{v_0, v_{n+1}\}, \quad (14)$$

$$x_{ij\tau}, s_{ij}, y_i, z_i \in \{0, 1\}, \quad (15)$$

**Table 1**  
Notation used in the mathematical model of CTP-VC.

|                           |                                                                                                                                 |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Sets</b>               |                                                                                                                                 |
| $V$                       | Set of all the nodes of the graph, $V = \{v_0, v_1, \dots, v_{n+1}\}$                                                           |
| $E$                       | Set of all the edges of the graph, including the edges to $v_0$ and $v_{n+1}$                                                   |
| $N_i$                     | Set of indices for all the nodes adjacent to node $v_i$ , $N_i = \{j \mid (i, j) \in E, j \in V\}, i = 0, 1, \dots, n+1$        |
| $T$                       | Set of time steps, $T = \{T_0 + k\Delta T, k = 0, \dots, h\}$                                                                   |
| <b>Parameters</b>         |                                                                                                                                 |
| $n$                       | Number of nodes                                                                                                                 |
| $C_{ij}$                  | Cost associated with edge $(i, j) \in E$                                                                                        |
| $\hat{C}_i$               | Cost of staying at node $v_i$ per time step                                                                                     |
| $W_i$                     | importance of node $v_i$                                                                                                        |
| $D_{ij}$                  | The shortest distance between node $v_i$ and node $v_j$                                                                         |
| $sp$                      | Speed with which the vehicle moves                                                                                              |
| $T_{ij}$                  | The time it takes to move from node $v_i$ to node $v_j$ , $T_{ij} = \lceil \frac{D_{ij}}{sp} \rceil$                            |
| $R$                       | Maximum coverage distance                                                                                                       |
| $\omega$                  | Scaling parameter corresponding to importance coverage in the objective function compared to the costs                          |
| $B$                       | Battery capacity of the vehicle                                                                                                 |
| $U$                       | Battery usage of the vehicle per kilometer while moving                                                                         |
| $\hat{U}$                 | Battery usage of the vehicle per time step while staying in a location                                                          |
| $M_1$                     | A large enough number, $M_1 \geq \max\{D_{ij}, (i, j) \in E\}$                                                                  |
| $M_2$                     | A large enough number, $M_2 \geq n - 1$                                                                                         |
| <b>Decision Variables</b> |                                                                                                                                 |
| $x_{ij\tau}$              | Binary variable; 1 if the vehicle starts moving from node $i$ to node $j$ , $(i, j) \in E$ , at time step $\tau$ ; 0 otherwise. |
| $y_i$                     | Binary variable; 1 if node $v_i$ is directly visited by the vehicle; 0 otherwise.                                               |
| $s_{ij}$                  | Binary variable; 1 if node $v_i$ is covered by node $v_j$ ; 0 otherwise. $((i, j) \in E)$                                       |
| $z_i$                     | Binary variable; 1 if node $v_i$ is indirectly covered by staying in another node; 0 otherwise.                                 |
| $r_i$                     | The radius of coverage for node $v_i$ as a function of the duration of stay at the node. $r_i = r(t_i)$                         |

**Table 2**  
Parameter values for Instance sets 1 and 2.

| Parameter name      | Value/range in Instance set 1        | Value/range in Instance set 2        |
|---------------------|--------------------------------------|--------------------------------------|
| Nodes location area | $50 \times 50$                       | $50 \times 50$                       |
| # of nodes ( $n$ )  | 5, 6, ..., 20                        | 50, 100, 150, 200, 250               |
| $\hat{C}_i$         | Uniformly at random from (0, 10)     | Uniformly at random from (0, 10)     |
| $W_i$               | Random integer between 1 and 100     | Random integer between 1 and 100     |
| $C_{ij} = D_{ij}$   | Euclidean distance between two nodes | Euclidean distance between two nodes |
| $\alpha$            | 1.5                                  | 1.5                                  |
| $\beta$             | 0                                    | 0                                    |
| $R$                 | 15                                   | 15                                   |
| $B$                 | 400                                  | 400                                  |
| $sp$                | 1                                    | 1                                    |
| $\omega$            | 1                                    | 1                                    |
| $U$                 | 2                                    | 2                                    |
| $\hat{U}$           | 1                                    | 1                                    |

The objective function in Eq. (2) quantifies the total gain associated with the nodes visited or covered, including costs associated with these operations. Constraints (4) and (5) ensure that the route commences and concludes at specified nodes. Constraint (6) enforces non-negative duration of stay at each node, as defined in Eq. (3), while constraint (7) restricts the vehicle to one activity per time step. Constraint (8) guarantees that the vehicle exits a node after entering it. To prevent exceeding the vehicle's battery capacity Eq. (9) is introduced. Furthermore, to ensure that no node is visited more than once, we enforce  $\sum_{i \in N_j} \sum_{t \in T} x_{ijt} \leq 1$  for  $j \in V \setminus \{v_0, v_{n+1}\}$ . This condition is inherently satisfied by defining the binary variable  $y_i$ . Constraint (10) stipulates that a node  $v_j$  is visited when the vehicle moves from an adjacent node to  $v_j$  at any time step. Constraint (11) ensures that if  $s_{ij} = 1$ , the node  $i$  must be covered by node  $j$ . Conversely, when node  $i$  is within the coverage radius of another node, the value of  $s_{ij}$  is contingent on the value of  $y_i$ . If the node is visited, its importance is accounted for in the objective function, resulting in  $s_{ij} = 0$ . Otherwise, the objective function incentivizes  $s_{ij}$  to take the value 1. Constraints (12) and (13) ensure that  $z_i = 1$  if there exists at least one  $j \in V$  for which  $s_{ij} = 1$ . Constraint (14) prevents double-counting the importance of a node when it is both visited and covered by another node. Finally, Eq. (15) specifies that all decision variables are binary.

#### 4. Solution methods and computational setup

In this section, we describe different methods that were used to solve the problem and their components. All the methods share the same set of heuristics defined in Section 4.2 and other configurations, i.e., initial solution, acceptance criteria, and score system. The only difference is the selection method for these heuristics.

##### 4.1. Instance sets

In this section, we describe the instance sets used in this computational study. Our instances are based on Instance sets 1 and 2 described in Torabi et al. (2023). In particular, the test instance sets are the exact instances from this study that we have also used as benchmarks in part of computational studies.

Based on Instance set 1, we have generated instances with specifications described in Table 2. The generated instances are split into  $N_t^1 = 3200$  training instances (200 instances per size) and  $N_v^1 = 800$  validation instances (50 instances per size). The test instance set comprises benchmark instances and includes  $N_s^1 = 80$  instances.

Similarly, based on Instance set 2, instances are generated according to the parameters in Table 2 and are split into  $N_t^2 = 1000$  training instances (200 instances per size) and  $N_v^2 = 250$  validation instances (50 instances per size). The benchmark Instance set 2 is used as test instance set and consists of  $N_s^2 = 25$  instances.

**Table 3**

List of removal operators.  $n$  indicates the total number of nodes in an instance.

| Operator name            | Description                                         |
|--------------------------|-----------------------------------------------------|
| <i>Random_remove_XXS</i> | $[0.02n] \leq q \leq \max\{2, [0.05n]\}$            |
| <i>Random_remove_XS</i>  | $[0.06n] \leq q \leq \max\{3, [0.1n]\}$             |
| <i>Random_remove_S</i>   | $[0.11n] \leq q \leq \max\{4, [0.15n]\}$            |
| <i>Random_remove_M</i>   | $\max\{2, [0.16n]\} \leq q \leq \max\{3, [0.2n]\}$  |
| <i>Random_remove_L</i>   | $\max\{2, [0.21n]\} \leq q \leq \max\{4, [0.23n]\}$ |
| <i>Random_remove_XL</i>  | $\max\{2, [0.24n]\} \leq q \leq \max\{5, [0.27n]\}$ |
| <i>Random_remove_XXL</i> | $\max\{3, [0.28n]\} \leq q \leq \max\{5, [0.3n]\}$  |

**Table 4**

Description of the insertion operator.

| Operator name                      | Description                                                                                                                                                                                                                                                                                                               |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Insert_70pc_greedy_duration</i> | The removed elements are inserted one by one in the positions that can improve the objective value the most, or cause the least possible cost. Furthermore, with the probability of 70%, the duration of stay in nodes is set to the most desirable value greedily, and in 30% of the cases, the duration is set to zero. |

#### 4.2. Heuristics

The heuristics used in this study combine a removal and an insertion operator. In each removal operator,  $q$  elements are randomly removed from the solution. The value of  $q$  is randomly selected from an interval specified in Table 3. We aim for  $q$  to be between one and 30% of  $n$ , dividing this range into seven segments, as shown in Table 3. For smaller instances, we use the maximum of the specific percentage of nodes and a constant number given in the table.

The insertion operator is described in Table 4. This is a typical insertion routine for routing problems but includes a segment for the duration of stay at each node. It is a hybrid greedy insertion: with the probability of 0.7, the most beneficial duration of stay for each inserted node is calculated, while in the remaining cases, this value is set to zero.

Moreover, the escape algorithm defined in Torabi et al. (2023) is utilized to prevent getting stuck in local optima, whenever the incumbent solution has not changed for more than 500 iterations. This escape algorithm adds  $2 \leq q \leq 6$  nodes to the visiting path, in the best possible positions, and with a duration of stay of 0.

#### 4.3. Adaptive Metaheuristic (AMH)

Here we have built upon the approach developed by Torabi et al. (2023), with a few tweaks to improve its performance. We conduct a full-factorial experiment, testing every possible combination of the initial solution, acceptance criteria, and score system, given below.

##### 1. Initial solution

We can set the initial solution as either of the following options.

- **Clustering method:** This algorithm creates an initial feasible solution using the k-means clustering. Nodes are clustered, and the ones closest to the cluster centers are added to the visiting path, subject to feasibility constraints. Initially, the duration of stay at these nodes is set to zero. The number of clusters,  $k$ , is determined using silhouette analysis, which identifies the optimal number of clusters within the range of two to  $\min\{n, 10\}$ .
- **LKH & Radius Improvement:** This algorithm first addresses the Traveling Salesman Problem (TSP) using the Lin-Kernighan heuristic (Lin and Kernighan, 1973), as implemented by Keld Helsgaun (LKH) (Helsgaun, 2000). Subsequently, the algorithm optimizes the duration of stay at each node by iterating through all nodes in the TSP

**Table 5**

Different scoring systems tested.

| Score system      | Best updated | Incumbent updated | New solution accepted | New feasible solution found | Neither |
|-------------------|--------------|-------------------|-----------------------|-----------------------------|---------|
| $S_{best}^{4210}$ | 4            | –                 | 2                     | 1                           | 0       |
| $S_{inc}^{4210}$  | –            | 4                 | 2                     | 1                           | 0       |
| $S^{6420}$        | 6            | 4                 | 2                     | –                           | 0       |
| $S^{8420}$        | 8            | 4                 | 2                     | –                           | 0       |

tour. This optimization seeks to remove certain nodes from the tour while covering them at the lowest possible cost. The algorithm ensures solution feasibility by considering factors such as battery capacity and avoiding unnecessary node coverage.

##### 2. Acceptance criteria

We take into account two different acceptance criteria, both variants of the Linear Record-to-Record Travel (RRT) acceptance criterion introduced by Dueck (1993), and given in Algorithm 1. This choice is due to promising results given by RRT when compared to other criteria, such as Simulated Annealing or Threshold Acceptance (Santini et al., 2018; Hemmati and Hvattum, 2017). Let  $x$ ,  $x_{best}$ , and  $x'$  be the incumbent solution, best-found solution, and the new solution given by applying a heuristic, respectively.

- **RRT<sub>best</sub>** compares the new objective value with the best-found, as the one shown in Algorithm 1. A new solution,  $x'$ , should have a gap of less than  $T$  with the best-found objective function,  $f(x_{best})$ .
- **RRT<sub>inc</sub>** compares the new objective value with the incumbent objective value, i.e., the new solution,  $x'$ , should have a gap of less than  $T$  with the incumbent objective function,  $f(x)$ .

**Algorithm 1** Record-to-Record Travel acceptance criterion. Santini et al. (2018)

**Require:**  $x_0$ , Initial solution, and  $T$ , initial threshold

**Ensure:**  $x_{best}$ , the best-found solution

```

 $x \leftarrow x_0$ 
 $x_{best} \leftarrow x, f(x_{best}) \leftarrow f(x)$ 
 $i \leftarrow 1$ 
while  $i \leq K$  do
  Pick  $x' \in N(x)$ 
  if  $\frac{f(x') - f(x_{best})}{f(x')} < T$  then
     $x \leftarrow x'$ 
  end if
  if  $f(x) < f(x_{best})$  then
     $x_{best} \leftarrow x$ 
  end if
  Update  $T$ 
   $i \leftarrow i + 1$ 
end while

```

##### 3. Score system

The fashion in which we evaluate the performance of each sub-heuristic determines the probability of choosing it in the subsequent iteration segments and therefore the overall performance of AMH. Thus we consider four alternatives for this choice in our analysis, shown in Table 5.

These experiments reveal that the best combination of initial solution, acceptance criteria, and scoring system are clustered solution, RRT<sub>best</sub>, and  $S^{8420}$ , respectively. This is an interesting observation in terms of the initial solution since it confirms a premature convergence when the algorithm initiates with a rather good solution.

**Table 6**

List of state features for DRLH used in this study.

| Name                  | Description                                                                       |
|-----------------------|-----------------------------------------------------------------------------------|
| <i>reduced_cost</i>   | The difference of objective value between previous & current solutions            |
| <i>cost_from_min</i>  | The difference of objective value between current & best found solution           |
| <i>cost</i>           | The objective value of the current solution                                       |
| <i>min_cost</i>       | The objective value of the best found solution                                    |
| <i>no_improvement</i> | The number of iterations since the last improvement                               |
| <i>index_step</i>     | The iteration number                                                              |
| <i>was_changed</i>    | 1 if the solution was changed from the previous, 0 otherwise.                     |
| <i>unseen</i>         | 1 if the solution has not previously been encountered in the search, 0 otherwise. |
| <i>last_action</i>    | The action in the previous iteration encoded in 1-hot.                            |

#### 4.4. Deep Reinforcement Learning Hyperheuristic (DRLH)

Here we briefly describe the DRLH framework proposed in Kallestad et al. (2023). A pseudo-code of this algorithm is given in Algorithm 2. DRLH shares a great deal with ALNS. The main deviation of this algorithm and ALNS is the probability of choosing each heuristic at each step, here denoted as a parameterized stochastic policy of the RL agent,  $\pi$ , which is defined as

$$\pi(h|s, \theta) = \Pr\{A_t = h \mid S_t = s, \theta_t = \theta\},$$

where  $\pi(h|s, \theta)$  is the probability of selecting heuristic  $h$  at step  $t$  with state  $s$ , the information that the environment shares with the agent, and  $\theta$  is the value of parameters in step  $t$ . In DRLH,  $\theta$  is the vector of weights in the MLP that is the agent. The MLP is trained using the policy gradient method of Proximal Policy Optimization (Schulman et al., 2017). We have used a subset of the state variables used in Kallestad et al. (2023) which are given in Table 6.

#### Algorithm 2 DRLH (Kallestad et al., 2023)

**Require:**  $H$ , a set of sub-heuristics  
**Ensure:**  $x_{best}$ , the best-found solution  
 Generate an initial solution  $x$  with objective function  $f(x)$   
 $x_{best} \leftarrow x, f(x_{best}) \leftarrow f(x)$   
**repeat**  
    $x' \leftarrow x$   
   Choose  $h \in H$  based on policy  $\pi(h|s, \theta)$   
   Apply heuristic  $h$  to  $x'$   
   **if**  $f(x') < f(x_{best})$  **then**  
      $x_{best} \leftarrow x$   
   **end if**  
   **if**  $\text{accept}(x', x)$  **then**  
      $x \leftarrow x'$   
   **end if**  
**until** stopping criteria met

##### 4.4.1. Training the agent

The agent is trained on each training instance set described in Section 4.1. The training is performed according to Algorithm 3, with hyperparameters set as follows: Learning rate 0.0005, Batch size 64, Hidden layer sizes 256, and Discount factor 0.5.

##### 4.4.2. Validation and model selection

In the training process, the agent adjusts its policy parameters after each episode, i.e., after an instance is solved. The trained model is saved after every 100 episodes, and among the saved models, one needs to be selected for testing.

We have made this selection using a standard validation step in machine learning practices. All validation instances are solved using URS to form a baseline. The 32 models saved for set 1 and 10 models saved for set 2 are run to solve validation Instance sets 1 and 2,

#### Algorithm 3 Training the Deep RL agent

**Ensure:**  $\pi^*$  optimal policy  
 Start with random setting of  $\theta$  for a random policy  $\pi$   
**for**  $e \leftarrow 1$  **to** *episodes* **do**  
   Receive initial state  $S_1$ ;  
   **for**  $t \leftarrow 1$  **to** *steps* **do**  
     Choose and perform action  $a \in A_t$  according to  $\pi(a|s, \theta)$   
     Receive  $R_t = v$  and  $s \in S_{t+1}$  from the environment  
   **end for**  
   Optimize the policy parameters,  $\theta$ , according to PPO (Schulman et al., 2017)  
**end for**

**Table 7**

Average GBK for AMH, URS, and DRLH.

| Instance set | Size | URS          | AMH          | DRLH         |
|--------------|------|--------------|--------------|--------------|
| 1            | 5    | <b>0.00%</b> | <b>0.00%</b> | <b>0.00%</b> |
|              | 6    | <b>0.00%</b> | <b>0.00%</b> | <b>0.00%</b> |
|              | 7    | <b>0.00%</b> | <b>0.00%</b> | <b>0.00%</b> |
|              | 8    | <b>0.00%</b> | <b>0.00%</b> | <b>0.00%</b> |
|              | 9    | <b>0.00%</b> | <b>0.00%</b> | <b>0.00%</b> |
|              | 10   | 0.17%        | 0.18%        | <b>0.14%</b> |
|              | 11   | 0.29%        | 0.37%        | <b>0.11%</b> |
|              | 12   | 0.97%        | 0.68%        | <b>0.44%</b> |
|              | 13   | 0.90%        | 0.87%        | <b>0.40%</b> |
|              | 14   | 0.96%        | 0.83%        | <b>0.27%</b> |
|              | 15   | 0.80%        | 0.68%        | <b>0.44%</b> |
|              | 16   | 0.45%        | 0.47%        | <b>0.14%</b> |
|              | 17   | 0.64%        | 0.69%        | <b>0.42%</b> |
|              | 18   | 2.42%        | 2.43%        | <b>1.78%</b> |
|              | 19   | 1.42%        | 1.54%        | <b>0.90%</b> |
|              | 20   | 1.07%        | 0.94%        | <b>0.47%</b> |
| 2            | 50   | 1.69%        | 1.99%        | <b>1.23%</b> |
|              | 100  | 2.45%        | 3.15%        | <b>1.33%</b> |
|              | 150  | 1.85%        | 2.42%        | <b>1.16%</b> |
|              | 200  | 1.74%        | 2.89%        | <b>1.42%</b> |
|              | 250  | 3.60%        | 3.29%        | <b>1.65%</b> |

respectively. Finally, the results yielded from each trained model on the validation set are compared to the URS baseline, and based on the improvement over baseline, the best model is selected for testing. The selected models for sets 1 and 2 are used to perform test experiments in Section 5.1.

#### 4.5. Uniform Random Selection (URS)

In this method, the heuristics are selected uniformly at random at each iteration. This is equivalent to setting the reaction factor,  $r$ , to zero in AMH.

### 5. Results and discussion

#### 5.1. Performance of selection methods

The different selection methods described in Sections 4.3–4.5 are all used with the heuristics introduced in Section 4.2 on the test instances, with 10 different random starting points. Each instance is solved for a total of 30 times, and the best-known solution,  $x_{bk}$ , is collected. The gap to best known, GBK, is defined as  $\text{GBK} = -(f(x_s) - f(x_{bk})) / f(x_{bk})$ , where  $f$  is the objective function evaluated in the solution  $x_s$  of a single run of each algorithm, and  $f(x_{bk})$  is the best-known objective value. For each selection method, the average of GBK is calculated over 10 runs and 5 instances of each size of each instance set. These average values are shown in Table 7, with the smallest value of GBK shown in boldface.

While for the 5 smallest size instances, all methods have reached the same results in all runs, when increasing the size, the DRLH selection

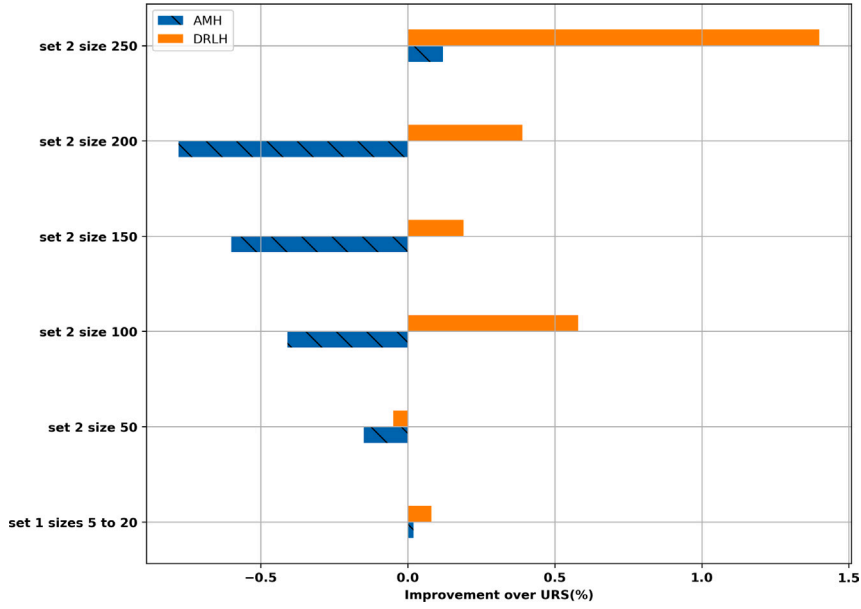


Fig. 1. Average Improvement on Uniform Random Search for each size category.

Table 8

Average improvements when using DRLH and the heuristics in this study compared to Torabi et al. (2023).

| Instance set | Size    | Average improvement |
|--------------|---------|---------------------|
| 1            | 5 to 20 | 0.01%               |
|              | 50      | 0.11%               |
|              | 100     | 2.14%               |
| 2            | 150     | 1.28%               |
|              | 200     | 0.68%               |
|              | 250     | 1.05%               |

method results in a lower average GBK which indicates the robustness of this method. It is also noteworthy to mention that no absolute superiority is observed when comparing AMH to URS, which indicates the poor selection capability of AMH.

Since URS uses an entirely randomized fashion to select heuristics at each iteration, the results obtained from it can be considered a baseline for other selection methods. Thus, we have computed the improvement over baseline for each run of DRLH and AMH and taken the average over all values yielded by each method for each size category, please see Fig. 1. The *size categories* in this study are defined as all the instances in set 1 (5–20), and each size in set 2 (50, 100, 150, 200, and 250). We can observe that for all size categories, the average improvements over URS are larger for DRLH than the corresponding value for AMH. Moreover, we observe that as a general rule, the better performance of DRLH over AMH increases with increasing size. This shows a great advantage for DRLH when we are dealing with larger-size problems, which are found more abundantly in real-world applications.

## 5.2. Solution quality of DRLH

In this study, we have used different, less sophisticated heuristics than the heuristics introduced by Torabi et al. (2023). Here we compare the overall solution quality obtained in this study using DRLH and the results obtained when using AMH on the heuristics introduced in Torabi et al. (2023) on the test instance sets. To that end, the improvement in the objective value is computed for each instance as  $(f(x^{\text{DRLH}}) - f(x^{\text{AMH}}))/f(x^{\text{AMH}})$ , where  $x^{\text{DRLH}}$  and  $x^{\text{AMH}}$  are the best solutions obtained by DRLH and AMH, respectively, and  $f$  computes the objective value.

In Table 8 the average improvement over each size category is reported. We can observe that in all cases, not only is the best-known

objective value reached using DRLH and the heuristics in this study, but also reveals a 0.26% improvement of the objective value when taking the average on all instances. In Instance set 2, the improvements are more substantial. This is to be expected since we allow the removal of up to 30% of the nodes in heuristics, which gives significantly more flexibility to explore the solution space for the larger-scale instances of set 2.

## 5.3. Robustness of selection methods

In this section, we aim to investigate the robustness of the selection methods in this study. This can be interpreted as the sensitivity of the best-found solution to the randomness embedded within the algorithm and selection methods. To that end, we compute the standard deviation of the values yielded by each method for each instance (10 values per method) and take the standard deviation of these values. When averaged over each size category, the results can be seen in Fig. 2. It is noteworthy that DRLH yields the results with the smallest average standard deviation among the three methods, for all the size categories. However, URS and AMH alternate in comparison, depending on the size category.

## 5.4. Heuristic selection frequency

The selection capability of each method is reflected in the frequency with which each heuristic is selected to act upon the incumbent solution. In Fig. 3 we can see the number of times each heuristic was selected in total, for 10 runs of 1000 iterations, when solving 5 different instances in set 2. We can see that Fig. 3(a) and Fig. 3(b) show very similar behavior regarding the frequency of usage of the heuristics, while Fig. 3(c) indicates a distinctive selection mechanism for the same instances.

## 5.5. Solution convergence

In this section, we consider the convergence of the three selection methods. To this end, the average objective value over all instances in each set is given for each iteration in Fig. 4. We observe that for test Instance set 1, the average behavior of the three selection methods is very similar. However, when zooming in after iteration 100, it can be seen that the values for the DRLH method are slightly above the other

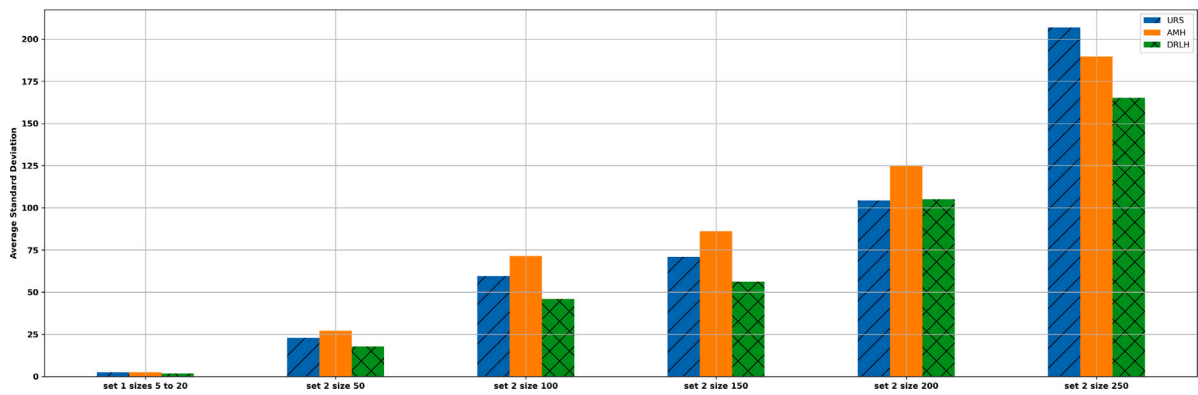


Fig. 2. Average standard deviation among the 10 different runs of each selection method.

two methods which indicate the slightly better solutions reached by this method for Instance set 1. Similarly, the average behavior of the three methods for Instance set 2 follows the same trend. It can be seen that while DRLH converges to a final solution in later iterations than URS and AMH, it reaches better solutions on average.

## 6. Conclusion

In this study, we have presented a comprehensive investigation into addressing the Covering Tour Problem with Varying Coverage (CTP-VC), leveraging a Deep Reinforcement Learning Hyperheuristic (DRLH) approach. Through exploration of different metaheuristics, namely Uniform Random Selection (URS), adaptive Metaheuristic (AMH), and the proposed DRLH method, we have shed light on the efficacy and potential of employing deep reinforcement learning techniques for tackling complex combinatorial optimization challenges. Our computational experiments, conducted with diverse instance sets and rigorous validation protocols, have provided valuable insights into the performance, solution quality, robustness, heuristic selection frequency, and solution convergence aspects of the proposed approach.

The results indicate that the DRLH approach exhibits competitive performance to the existing metaheuristic (Torabi et al., 2023), yielding high-quality solutions while maintaining robustness across different problem instances. Furthermore, the results show that the superiority of solutions given by DRLH is more prominent when the instance size increases, which makes DRLH more favorable for solving larger-size instances. The analysis of heuristic selection frequency reveals the adaptability of the DRLH agent in dynamically selecting appropriate heuristics based on the problem characteristics. Additionally, the observed solution convergence highlights the efficacy of the DRLH approach in achieving good solutions within reasonable computational time frames.

The value of DRLH lies in its ability to learn from solving problems repeatedly during the training phase. Unlike algorithms such as ALNS which encounter each instance with no prior knowledge, DRLH uses a trained model to make informed heuristic choices. This model contains valuable information about heuristic selection, leading to a more efficient solving procedure for new problem instances. While the complexity of the method increases because of the training and validation phases, these stages need to be completed only once for each problem set. Consequently, we believe the increased complexity is justified by the method's enhanced solution quality and robustness. This approach ensures that once trained, DRLH can consistently deliver high-quality, robust solutions across various problem instances, making it a highly effective tool for tackling combinatorial optimization challenges.

While our findings highlight the importance of incorporating deep reinforcement learning techniques into hyperheuristic methodologies to tackle complex combinatorial optimization problems like the CTP-VC, it is important to acknowledge the limitations of the conclusions drawn

from this study. We have emphasized the power of DRLH in different respects when solving CTP-VC, and while this is a testament to the general DRLH introduced in Kallestad et al. (2023), it is not conclusive proof of the superiority of this method to conventional methods. More experiments on other problem types could be conducted to explore the versatility and applicability of DRLH across diverse domains.

Other research avenues could be explored in future studies. Among others, the integration of domain-specific knowledge could be advantageous. For instance, problem-specific features can be incorporated into the DRLH framework as state variables to enhance its adaptability and effectiveness in solving real-world instances of the CTP-VC. Moreover, empirical studies can be conducted to assess the practical scalability of the DRLH approach on large-scale and real-world instances of the CTP-VC. By pursuing these future directions, researchers can advance the state-of-the-art in hyperheuristic optimization techniques, contribute to the development of efficient and adaptive algorithms for complex optimization problems, and pave the way for practical applications in various fields.

## CRediT authorship contribution statement

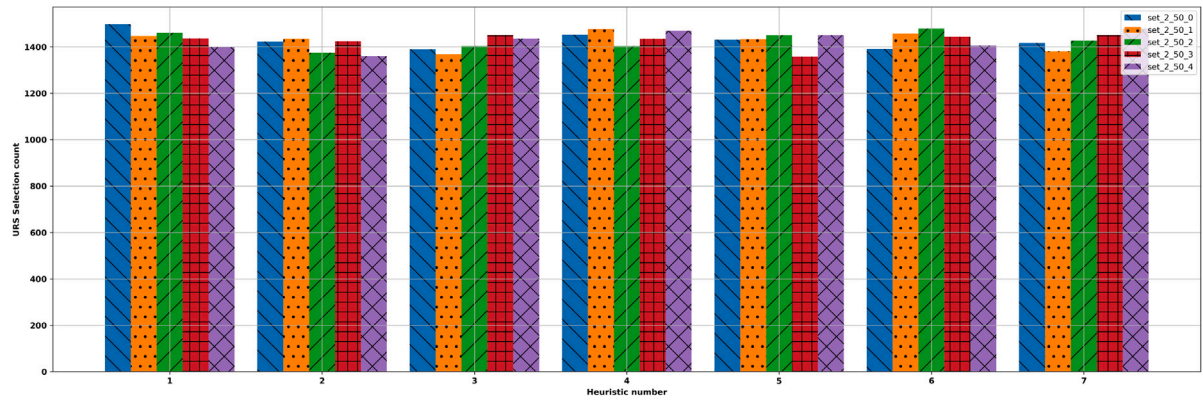
**Parisa Torabi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Formal analysis, Data curation, Conceptualization. **Ahmad Hemmati:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Anna Oleynik:** Writing – review & editing, Supervision, Investigation, Formal analysis. **Guttorm Alendal:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition.

## Acknowledgments

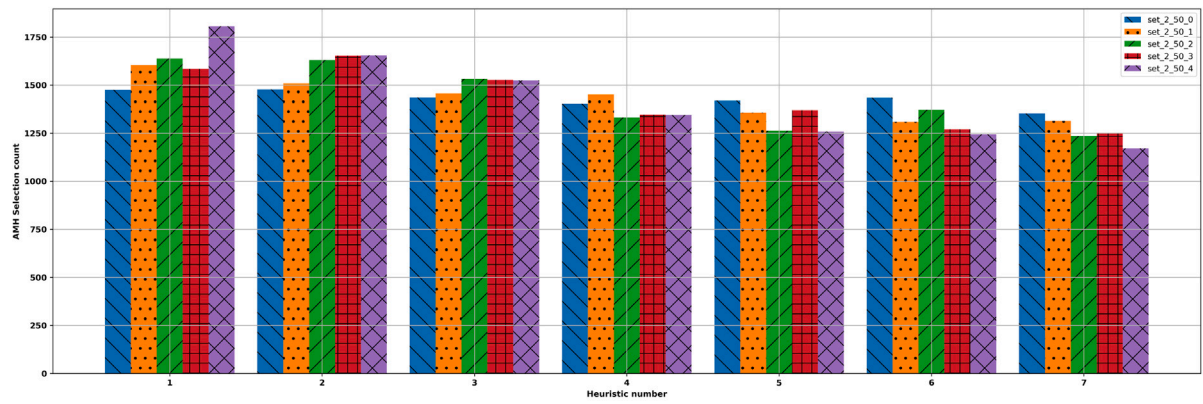
This work is part of ACTOM, a project that is funded through the ACT programme (Accelerating CCS Technologies, Horizon2020 Project No 294766). Financial contributions made from; The Research Council of Norway, (RCN), Norway, Ministry of Economic Affairs and Climate Policy, the Netherlands, Department for Business, Energy & Industrial Strategy (BEIS), United Kingdom together with extra funding from NERC and EPSRC research councils, United Kingdom, US-Department of Energy (US-DOE), USA. In-kind contributions from the University of Bergen are gratefully acknowledged. Anna Oleynik is funded through the Academia agreement between Equinor and the University of Bergen. Ahmad Hemmati's contribution to the work has been through the Ocean Charger project funded by the Norwegian Research Council (Project No. 340936).

## Data availability

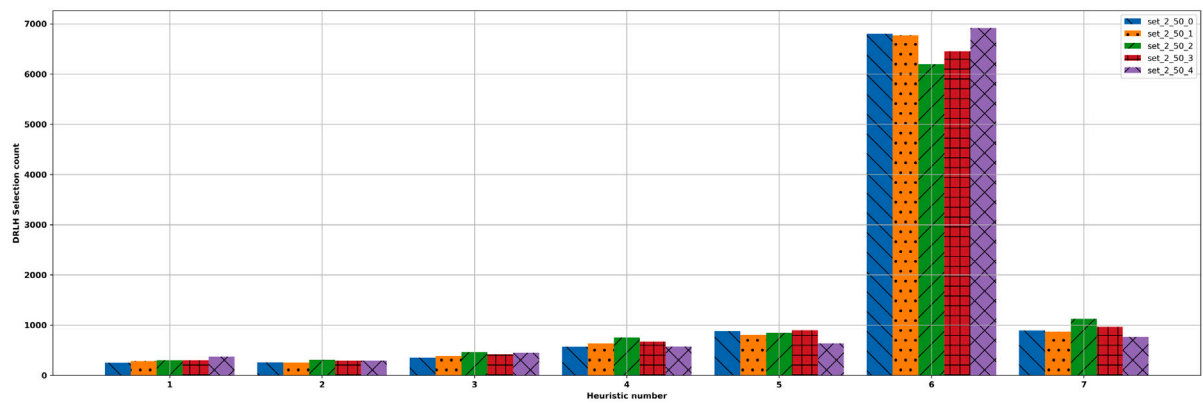
Data will be made available on request.



(a) Uniform Random Search

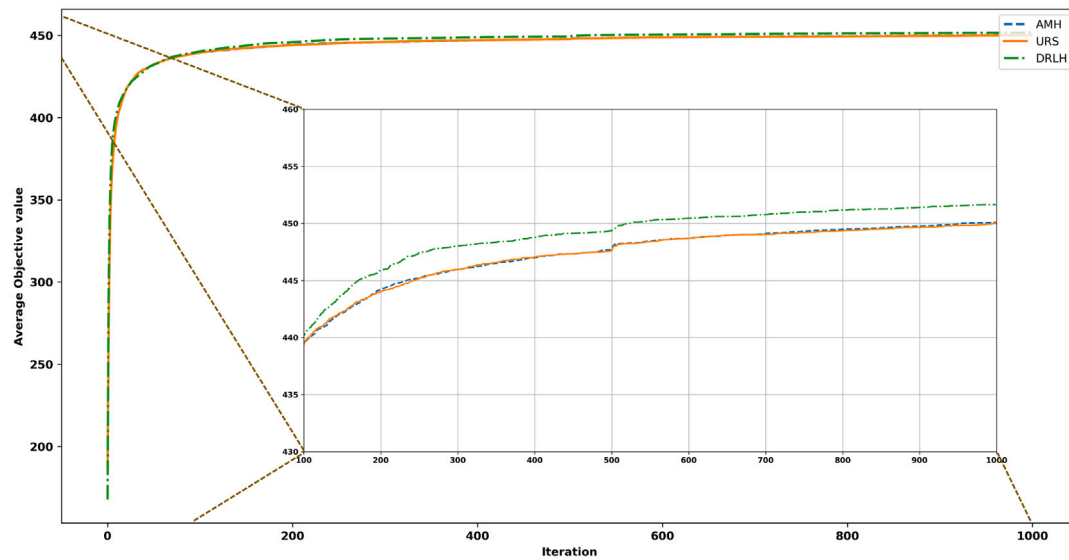


(b) Adaptive Meta-Heuristic

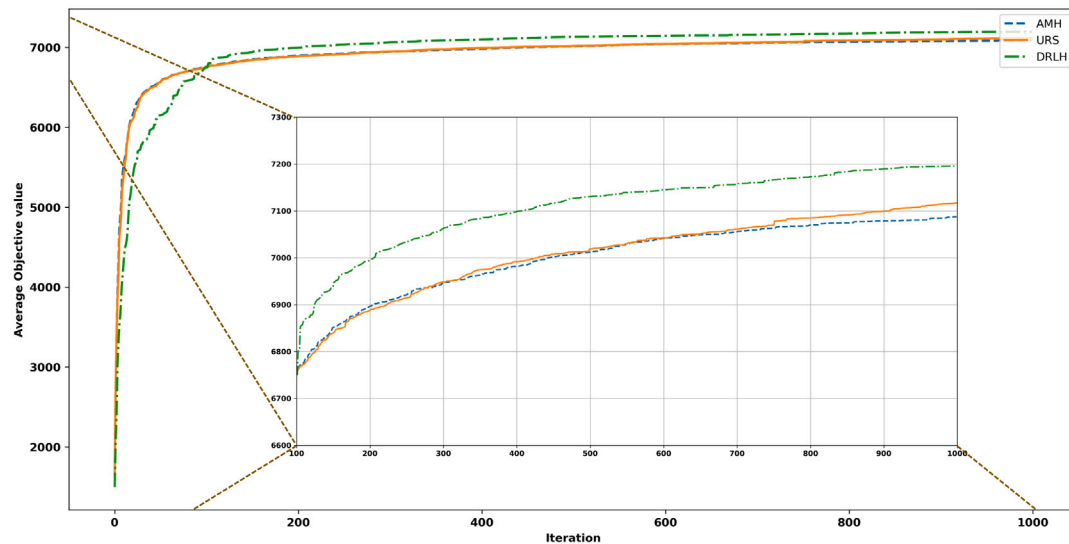


(c) Deep Reinforcement Learning Hyperheuristic

Fig. 3. Frequency of selection for each heuristic when using URS, AMH, and DRLH, shown for a subset of instances.



(a) Average over set 1



(b) Average over set 2

Fig. 4. Average of the best-found objective value at each iteration for AMH, URS, and DRLH.

## References

- Asta, Shahriar, Ozcan, Ender, 2014. An apprenticeship learning hyper-heuristic for vehicle routing in HyFlex. In: 2014 IEEE Symposium on Evolving and Autonomous Learning Systems (EALS). IEEE, Orlando, FL, USA, ISBN: 978-1-4799-4494-1, pp. 65–72. <http://dx.doi.org/10.1109/EALS.2014.7009505>, URL <http://ieeexplore.ieee.org/document/7009505/>.
- Baldacci, Roberto, Boschetti, Marco A, Maniezzo, Vittorio, Zamboni, Marco, 2005. Scatter search methods for the covering tour problem. *Metaheuristic Optim. Mem. Evol. Tabu Search Scatter Search* 59–91.
- Bello, Irwan, Pham, Hieu, Le, Quoc V., Norouzi, Mohammad, Bengio, Samy, 2016. Neural combinatorial optimization with reinforcement learning. <http://dx.doi.org/10.48550/ARXIV.1611.09940>, URL <https://arxiv.org/abs/1611.09940>. Publisher: arXiv Version Number: 3.
- Bengio, Yoshua, Lodi, Andrea, Prouvost, Antoine, 2021. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European J. Oper. Res.* (ISSN: 03772217) 290 (2), 405–421. <http://dx.doi.org/10.1016/j.ejor.2020.07.063>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0377221720306895>.
- Brown, M.L., Fintor, L., 1995. U.S. screening mammography services with mobile units: results from the national survey of mammography facilities. *Radiology* (ISSN: 0033-8419) 195 (2), 529–532. <http://dx.doi.org/10.1148/radiology.195.2.7724778>, URL <http://pubs.rsna.org/doi/10.1148/radiology.195.2.7724778>.
- Burke, Edmund K., Hyde, Matthew, Kendall, Graham, Ochoa, Gabriela, Özcan, Ender, Woodward, John R., 2010. A classification of hyper-heuristic approaches. In: Gendreau, Michel, Potvin, Jean-Yves (Eds.), *Handbook of Metaheuristics*. In: International Series in Operations Research & Management Science, vol. 146, Springer US, ISBN: 978-1-4419-1663-1 978-1-4419-1665-5, pp. 449–468. [http://dx.doi.org/10.1007/978-1-4419-1665-5\\_15](http://dx.doi.org/10.1007/978-1-4419-1665-5_15), URL [https://link.springer.com/10.1007/978-1-4419-1665-5\\_15](https://link.springer.com/10.1007/978-1-4419-1665-5_15).
- Cowling, Peter, Kendall, Graham, Soubeiga, Eric, 2001. A hyperheuristic approach to scheduling a sales summit. In: Goos, Gerhard, Hartmanis, Juris, Van Leeuwen, Jan, Burke, Edmund, Erben, Wilhelm (Eds.), *Practice and Theory of Automated Timetabling III*. In: Lecture Notes in Computer Science, vol. 2079, Springer Berlin Heidelberg, ISBN: 978-3-540-42421-5 978-3-540-44629-3, pp. 176–190. [http://dx.doi.org/10.1007/3-540-44629-X\\_11](http://dx.doi.org/10.1007/3-540-44629-X_11), URL [http://link.springer.com/10.1007/3-540-44629-X\\_11](http://link.springer.com/10.1007/3-540-44629-X_11).
- Cubillos, Maximiliano, Wöhlk, Sanne, 2021. Solution of the maximal covering tour problem for locating recycling drop-off stations. *J. Oper. Res. Soc.* (ISSN: 0160-5682) 72 (8), 1898–1913. <http://dx.doi.org/10.1080/01605682.2020.1746701>, URL <https://www.tandfonline.com/doi/full/10.1080/01605682.2020.1746701>.
- Current, John R., Schilling, David A., 1989. The covering salesman problem. *Transp. Sci.* (ISSN: 0041-1655) 23 (3), 208–213. <http://dx.doi.org/10.1287/trsc.23.3.208>, URL <https://pubsonline.informs.org/doi/10.1287/trsc.23.3.208>.

- Current, John R., Schilling, David A., 1994. The median tour and maximal covering tour problems: Formulations and heuristics. *European J. Oper. Res.* (ISSN: 03772217) 73 (1), 114–126. [http://dx.doi.org/10.1016/0377-2217\(94\)90149-X](http://dx.doi.org/10.1016/0377-2217(94)90149-X), URL <https://linkinghub.elsevier.com/retrieve/pii/S037722179490149X>.
- Doerner, Karl, Focke, Axel, Gutjahr, Walter J., 2007. Multicriteria tour planning for mobile healthcare facilities in a developing country. *European J. Oper. Res.* (ISSN: 03772217) 179 (3), 1078–1096. <http://dx.doi.org/10.1016/j.ejor.2005.10.067>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0377221706000774>.
- Dueck, Gunter, 1993. New optimization heuristics. *J. Comput. Phys.* (ISSN: 00219991) 104 (1), 86–92. <http://dx.doi.org/10.1006/jcph.1993.1010>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0021999183710107>.
- Gendreau, Michel, Laporte, Gilbert, Semet, Frédéric, 1997. The covering tour problem. *Oper. Res.* 45 (4), 568–576, Publisher: Informa.
- Glize, Estèle, Roberti, Roberto, Jozefowicz, Nicolas, Nguveu, Sandra Ulrich, 2020. Exact methods for mono-objective and bi-objective multi-vehicle covering tour problems. *European J. Oper. Res.* (ISSN: 03772217) 283 (3), 812–824. <http://dx.doi.org/10.1016/j.ejor.2019.11.045>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0377221719309579>.
- Glover, Fred, 1977. Heuristics for integer programming using surrogate constraints. *Decis. Sci.* (ISSN: 0011-7315) 8 (1), 156–166. <http://dx.doi.org/10.1111/j.1540-5915.1977.tb01074.x>, URL <https://onlinelibrary.wiley.com/doi/10.1111/j.1540-5915.1977.tb01074.x>.
- Hachicha, Mondher, John Hodgson, M., Laporte, Gilbert, Semet, Frédéric, 2000. Heuristics for the multi-vehicle covering tour problem. *Comput. Oper. Res.* (ISSN: 03050548) 27 (1), 29–42. [http://dx.doi.org/10.1016/S0305-0548\(99\)00006-4](http://dx.doi.org/10.1016/S0305-0548(99)00006-4), URL <https://linkinghub.elsevier.com/retrieve/pii/S0305054899000064>.
- Helsgaun, Keld, 2000. An effective implementation of the Lin-Kernighan traveling salesman heuristic. *European J. Oper. Res.* 126 (1), 106–130, Publisher: Elsevier.
- Hemmati, Ahmad, Hvattum, Lars Magnus, 2017. Evaluating the importance of randomization in adaptive large neighborhood search. *Int. Trans. Oper. Res.* (ISSN: 09696016) 24 (5), 929–942. <http://dx.doi.org/10.1111/itor.12273>, URL <https://onlinelibrary.wiley.com/doi/10.1111/itor.12273>.
- Hodgson, M. John, Laporte, Gilbert, Semet, Frederic, 1998. A covering tour model for planning mobile health care facilities in SuhumDistrict, Ghana. *J. Reg. Sci.* (ISSN: 0022-4146) 38 (4), 621–638. <http://dx.doi.org/10.1111/0022-4146.00113>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/0022-4146.00113>.
- Jozefowicz, Nicolas, 2014. A branch-and-price algorithm for the multivehicle covering tour problem. *Networks* (ISSN: 00283045) 64 (3), 160–168. <http://dx.doi.org/10.1002/net.21564>, URL <https://onlinelibrary.wiley.com/doi/10.1002/net.21564>.
- Jozefowicz, Nicolas, Semet, Frédéric, Talbi, El-Ghazali, 2007. The bi-objective covering tour problem. *Comput. Oper. Res.* (ISSN: 03050548) 34 (7), 1929–1942. <http://dx.doi.org/10.1016/j.cor.2005.07.022>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0305054805002480>.
- Kallestad, Jakob, Hasibi, Ramin, Hemmati, Ahmad, Sörensen, Kenneth, 2023. A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. *European J. Oper. Res.* (ISSN: 03772217) 309 (1), 446–468. <http://dx.doi.org/10.1016/j.ejor.2023.01.017>, URL <https://linkinghub.elsevier.com/retrieve/pii/S037722172300036X>.
- Kool, Wouter, van Hoof, Herke, Welling, Max, 2019. Attention, learn to solve routing problems!. URL <http://arxiv.org/abs/1803.08475>. cs,stat.
- Kovács, Gyöngyi, Spens, Karen M., 2007. Humanitarian logistics in disaster relief operations. *Int. J. Phys. Distrib. Logist. Manage.* 37 (2), 99–114.
- Labbe, Martine, Laporte, Gilbert, 1986. Maximizing user convenience and postal service efficiency in post box location. *JORBEL-Belg. J. Oper. Res. Stat. Comput. Sci.* 26 (2), 21–36.
- Li, Kaiwen, Zhang, Tao, Wang, Rui Wang Yuheng, Han, Yi, 2021. Deep reinforcement learning for combinatorial optimization: Covering salesman problems. arXiv preprint [arXiv:2102.05875](https://arxiv.org/abs/2102.05875).
- Lin, Shen, Kernighan, Brian W., 1973. An effective heuristic algorithm for the traveling-salesman problem. *Oper. Res.* 21 (2), 498–516, Publisher: Informa.
- Lu, Hao, Zhang, Xingwen, Yang, Shuang, 2019. A learning-based iterative method for solving vehicle routing problems. In: *International Conference on Learning Representations*.
- Mazyavkina, Nina, Sviridov, Sergey, Ivanov, Sergei, Burnaev, Evgeny, 2021. Reinforcement learning for combinatorial optimization: A survey. *Comput. Oper. Res.* (ISSN: 03050548) 134, 105400. <http://dx.doi.org/10.1016/j.cor.2021.105400>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0305054821001660>.
- Nazari, MohammadReza, Oroojlooy, Afshin, Snyder, Lawrence, Takac, Martin, 2018. Reinforcement learning for solving the vehicle routing problem. In: Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (Eds.), *Advances in Neural Information Processing Systems*. vol. 31, Curran Associates, Inc., URL [https://proceedings.neurips.cc/paper\\_files/paper/2018/file/9fb4651c05b2ed70fba5afe0b039a550-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2018/file/9fb4651c05b2ed70fba5afe0b039a550-Paper.pdf).
- Oliveira, Washington Alves De, Moretti, Antonio Carlos, Reis, Ednei Felix, 2015. Multi-vehicle covering tour problem: BUILDING routes for urban patrolling. *Pesquisa Operacional* (ISSN: 0101-7438) 35 (3), 617–644. <http://dx.doi.org/10.1590/0101-7438.2015.035.03.0617>, URL [http://www.scielo.br/scielo.php?script=sci\\_arttext&pid=S0101-74382015000300617&lng=en&tlng=en](http://www.scielo.br/scielo.php?script=sci_arttext&pid=S0101-74382015000300617&lng=en&tlng=en).
- Oppong, Joseph R., Hodgson, M. John, 1994. Spatial accessibility to health care facilities in Suhum district, Ghana. *The Professional Geographer* (ISSN: 0033-0124) 46 (2), 199–209. <http://dx.doi.org/10.1111/j.0033-0124.1994.00199.x>, URL <http://www.tandfonline.com/doi/abs/10.1111/j.0033-0124.1994.00199.x>.
- Ouelmokhtar, Hand, Benmoussa, Yahia, Diguët, Jean-Philippe, Benazzouz, Djamel, Lemarchand, Laurent, 2022. Near-optimal covering solution for USV coastal monitoring using PAES. *J. Intell. Robot. Syst.* 106 (1), 24, Publisher: Springer.
- Özcan, Ender, Misir, Mustafa, Ochoa, Gabriela, Burke, Edmund K., 2012. A reinforcement learning: great-deluge hyper-heuristic for examination timetabling. In: *Modeling, Analysis, and Applications in Metaheuristic Computing: Advancements and Trends*. IGI Global, pp. 34–55.
- Santini, Alberto, Ropke, Stefan, Hvattum, Lars Magnus, 2018. A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. *J. Heuristics* (ISSN: 1381-1231) 24 (5), 783–815. <http://dx.doi.org/10.1007/s10732-018-9377-x>, URL <http://link.springer.com/10.1007/s10732-018-9377-x>.
- Schulman, John, Wolski, Filip, Dhariwal, Prafulla, Radford, Alec, Klimov, Oleg, 2017. Proximal policy optimization algorithms. URL <http://arxiv.org/abs/1707.06347>. cs.
- Simms, John C., 1989. Fixed and mobile facilities in dairy practice. *Vet. Clin. North Am. Food Animal Pract.* (ISSN: 07490720) 5 (3), 591–601. [http://dx.doi.org/10.1016/S0749-0720\(15\)30953-1](http://dx.doi.org/10.1016/S0749-0720(15)30953-1), URL <https://linkinghub.elsevier.com/retrieve/pii/S0749072015309531>.
- Stefan Ropke, David Pisinger, 2006. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transp. Sci.* 40 (4), 455–472. <http://dx.doi.org/10.1287/trsc.1050.0135>.
- Torabi, Parisa, Oleynik, Anna, Hemmati, Ahmad, Alendal, Guttorm, 2023. Covering tour problem with varying coverage: Application to marine environmental monitoring. *Appl. Math. Model.* <http://dx.doi.org/10.1016/j.apm.2023.07.024>.
- Tricoire, Fabien, Graf, Alexandra, Gutjahr, Walter J., 2012. The bi-objective stochastic covering tour problem. *Comput. Oper. Res.* (ISSN: 03050548) 39 (7), 1582–1592. <http://dx.doi.org/10.1016/j.cor.2011.09.009>, URL <https://linkinghub.elsevier.com/retrieve/pii/S030505481100270X>.
- Turkeş, Renata, Sörensen, Kenneth, Hvattum, Lars Magnus, 2021. Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. *European J. Oper. Res.* (ISSN: 03772217) 292 (2), 423–442. <http://dx.doi.org/10.1016/j.ejor.2020.10.045>, URL <https://linkinghub.elsevier.com/retrieve/pii/S037722172030936X>.
- Tyasnurita, Raras, Özcan, Ender, Shahriar, Asta, John, Robert, 2015. Improving performance of a hyper-heuristic using a multilayer perceptron for vehicle routing. 15th UK Workshop Comput. Intell. (UKCI 2015) URL <https://nottingham-repository.worktribe.com/output/761994>.
- Vesselinova, Natalia, Steinert, Rebecca, Perez-Ramirez, Daniel F., Boman, Magnus, 2020. Learning combinatorial optimization on graphs: A survey with applications to networking. *IEEE Access* (ISSN: 2169-3536) 8, 120388–120416. <http://dx.doi.org/10.1109/ACCESS.2020.3004964>, URL <https://ieeexplore.ieee.org/document/9125934/>.
- Wang, Qi, Tang, Chunlei, 2021. Deep reinforcement learning for transportation network combinatorial optimization: A survey. *Knowl.-Based Syst.* (ISSN: 09507051) 233, 107526. <http://dx.doi.org/10.1016/j.knsys.2021.107526>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0950705121007887>.
- Windras Mara, Setyo Tri, Norcahyo, Rachmadi, Jodiawan, Panca, Lusiantoro, Luluk, Rifai, Achmad Pratama, 2022. A survey of adaptive large neighborhood search algorithms and applications. *Comput. Oper. Res.* (ISSN: 03050548) 146, 105903. <http://dx.doi.org/10.1016/j.cor.2022.105903>, URL <https://linkinghub.elsevier.com/retrieve/pii/S0305054822001654>.
- Yang, Yunhao, Whinston, Andrew, 2020. A survey on reinforcement learning for combinatorial optimization. URL <http://arxiv.org/abs/2008.12248>. cs,stat.
- Yu, James J.Q., Yu, Wen, Gu, Jiatao, 2019. Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* (ISSN: 1524-9050) 20 (10), 3806–3817. <http://dx.doi.org/10.1109/ITITS.2019.2909109>, URL <https://ieeexplore.ieee.org/document/8693516/>.
- Zhou, Jie, Cui, Ganqu, Hu, Shengding, Zhang, Zhengyan, Yang, Cheng, Liu, Zhiyuan, Wang, Lifeng, Li, Changcheng, Sun, Maosong, 2021. Graph neural networks: A review of methods and applications. URL <http://arxiv.org/abs/1812.08434>. cs,stat.